Practical No. 1: * Write a program to display " Hello World" using:
console.log()
document.write( )
alert ()

IX. Conclusion

JavaScript is a powerful scripting language used to make web pages dynamic and interactive.
By using functions like `alert`, `document.write`, and `console.log`, we can display messages, format content, and debug programs easily.
Understanding the basics of JavaScript helps in creating responsive and user-friendly websites.

X. Practical Related Questions

1. Display the dialog box using `alert` to display multiline text.

```
<script>
alert("Welcome to JavaScript\nThis is a multiline message\nThank you!");
</script>
```

Explanation:
The `\n` is used to display text on a new line inside the alert dialog box.

---

2. Display the text in `document.write` using HTML formatting tags.

```
<script>
      document.write("<h2>JavaScript Output</h2>");
      document.write("<b>This text is bold</b><br>");
      document.write("<i>This text is italic</i>");
</script>
```

Explanation:
HTML formatting tags like `<h2>`, `<b>`, `<i>`, and `<br>` are used inside `document.write()` to format text.

XI. Exercise

1. Write steps or ways to observe the output of `console.log`.

Answer:

1. Write `console.log()` in the JavaScript code.
2. Open the webpage in a web browser.
3. Right-click on the page and choose **Inspect**.
4. Click on the **Console** tab.
5. The output will be displayed in the console window.

---

2. State the purpose of a scripting language like JavaScript.

Answer:

- To create interactive web pages
- To validate user input
- To perform calculations
- To respond to user events such as clicks and key presses
- To update webpage content dynamically

---

3. Write and explain the syntax of the `script` tag in HTML.

```
<script>
  // JavaScript code
</script>
```

Explanation:

- The `<script>` tag is used to insert JavaScript into an HTML document.
- JavaScript code is written between the opening and closing `<script>` tags.
- The script tag can be placed in the `<head>` or `<body>` section of HTML.

Practical No. 4:

1. Setup Angular development environment using

i) Installation of Node.js and npm ii) Installation of Angular CLI

2. Write a program to display "Good Morning" message on Webpage.

IX. Conclusion

Angular is a popular framework used for developing dynamic web applications.
By installing Node.js, npm, and Angular CLI, we can easily set up the Angular development environment.
Angular allows developers to create fast, structured, and interactive web pages such as displaying simple messages like "Good Morning".

X. Practical Related Questions

1. Develop app in AngularJS to display "Good Morning" message in browser.

Program:

```
<!DOCTYPE html>
<html>
<head>
  <title>Good Morning App</title>
  <!-- AngularJS CDN -->
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

<body ng-app="myApp" ng-controller="myCtrl">

  <h2> {{message}} </h2>

  <script>
    var app = angular.module("myApp", []);

    app.controller("myCtrl", function($scope) {
      $scope.message = "Good Morning,SPC";
    });
  </script>

</body>
</html>
```

Output:
The browser displays **Good Morning** message.

---

2. Create Angular App to display “Hello World” message.

Program:

```
<!-- app.component.html -->
<h1>Hello World</h1>
```

Output:
The browser displays **Hello World** message.

XI. Exercise

1. Give full form of npm and AngularJS CLI.

- **npm** – Node Package Manager
- **AngularJS CLI** – Angular Command Line Interface

---

2. Which file in the file directory of AngularJS will contain the code for output.

Answer:
The output code is written in:

```
app.component.html
```

---

3. List advantages of AngularJS.

Advantages of AngularJS:

- Supports MVC architecture
- Two-way data binding
- Faster and efficient development
- Easy to maintain and test
- Supports reusable components
- Suitable for single page applications

Practical No. 5: *Write AngularJs Program to design form using various controls and apply validations on inputs.

IX. Conclusion

AngularJS provides powerful features for creating dynamic and interactive web forms.
By using directives like `ng-model`, `ng-show`, `ng-controller`, and `ng-click`, form validation becomes easy and efficient.
AngularJS improves user experience by validating input fields and displaying error messages dynamically.

X. Practical Related Questions

1. Develop New Employee Registration Form with different form controls and validate it with `ng-show`.

```
<!DOCTYPE html>
<html ng-app="empApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="empCtrl">

<form name="empForm">
Employee Name:
<input type="text" name="ename" ng-model="emp.name" required>
<span ng-show="empForm.ename.$touched && empForm.ename.$invalid">
Name is required
</span><br><br>

Email:
<input type="email" name="email" ng-model="emp.email" required>
<span ng-show="empForm.email.$touched && empForm.email.$invalid">
Valid Email required
</span><br><br>

<input type="submit" value="Register">
</form>

<script>
angular.module("empApp",[])
.controller("empCtrl",function($scope){});
</script>

</body>
</html>
```

---

2. Develop Student Exam Registration Form and validate it with `ng-controller` directive.

```
<!DOCTYPE html>
<html ng-app="studApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="studCtrl">

<form name="studForm">
Student Name:
<input type="text" name="sname" ng-model="student.name" required>
<span ng-show="studForm.sname.$invalid && studForm.sname.$touched">
Name required
</span><br><br>

Roll No:
<input type="number" name="roll" ng-model="student.roll" required>
<span ng-show="studForm.roll.$invalid && studForm.roll.$touched">
Roll No required
</span><br><br>

<input type="submit" value="Submit">
</form>

<script>
angular.module("studApp",[])
.controller("studCtrl",function($scope){});
</script>

</body>
</html>
```

---

3. Develop a form with different controls and validate it on click of submit button using `ng-click`.

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="myCtrl">

<form>
Username:
<input type="text" ng-model="user.name"><br><br>

Password:
<input type="password" ng-model="user.pass"><br><br>

<button ng-click="validate()">Submit</button>
<p>{{message}}</p>
</form>
```

```
<script>
angular.module("myApp",[])
.controller("myCtrl",function($scope){
$scope.validate=function(){
if(!$scope.user || !$scope.user.name || !$scope.user.pass)
$scope.message="All fields are required";
else
$scope.message="Form submitted successfully";
}
});
</script>

</body>
</html>
```

XI. Exercise

1. State the use of `ng-model` and `name` attributes of Input field in AngularJS Form.

- **ng-model:** Binds the input field value to the model.
- **name:** Identifies the input field and helps track validation state.

---

2. Which directive will be used to display error messages.

Answer:
`ng-show` directive is used to display error messages.

---

3. List various validation states of input fields.

- `$valid`
- `$invalid`
- `$touched`
- `$untouched`
- `$dirty`
- `$pristine`

Practical No. 6: *Write a program to display data model view and display the data for given problem.

IX. Conclusion

AngularJS follows the **Model–View–Controller (MVC)** architecture, which helps in separating data, logic, and presentation.
Using data binding, AngularJS automatically updates the view when the model changes.
This makes it easy to display and manage dynamic data on a webpage.

X. Practical Related Questions

1. Display Hello <name>. Set the name as your name.

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="myCtrl">

<h2>Hello {{name}}</h2>

<script>
angular.module("myApp",[])
.controller("myCtrl",function($scope){
  $scope.name = "Student";
});
</script>

</body>
</html>
```

Output:
Hello Student

---

2. Set the first name and last name in separate property and display full name.

```
<!DOCTYPE html>
<html ng-app="nameApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="nameCtrl">

<h2>Full Name: {{firstName}} {{lastName}}</h2>
```

```
<script>
angular.module("nameApp",[])
.controller("nameCtrl",function($scope){
  $scope.firstName = "Rahul";
  $scope.lastName = "Sharma";
});
</script>

</body>
</html>
```

Output:
Full Name: Rahul Sharma

XI. Exercise

1. How do you display a value from your controller using one-way binding?

Answer: By using **interpolation (`{{ }}`)** in the HTML view to display values from the controller.

Example:

```
{{message}}
```

---

2. What is interpolation in data binding?

Answer: Interpolation is a technique in AngularJS where expressions written inside `{{ }}` are evaluated and the result is displayed in the view.

Example:

```
<h1>{{name}}</h1>
```

Practical No.7: Write a program to display two-way data binding.

IX. Conclusion

Two-way data binding is an important feature of AngularJS that keeps the **model and view synchronized**. When the user changes the data in the view, the model updates automatically, and when the model changes, the view updates instantly.
This reduces coding effort and makes web applications more interactive and user-friendly.

X. Practical Related Questions

1. Accept name in text field in the form and display it.

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="myCtrl">

Enter Name:
<input type="text" ng-model="name">

<h2>Hello {{name}}</h2>

<script>
angular.module("myApp",[])
.controller("myCtrl",function($scope){
  $scope.name = "";
});
</script>

</body>
</html>
```

Explanation:
The `ng-model` directive binds the input field to the model, enabling two-way data binding.

---

2. Create a registration form. On clicking submit, display the entered details.

```
<!DOCTYPE html>
<html ng-app="regApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="regCtrl">

<form>
Name:
```

```
<input type="text" ng-model="user.name"><br><br>

Email:
<input type="email" ng-model="user.email"><br><br>

<button ng-click="show=true">Submit</button>
</form>

<div ng-show="show">
  <h3>Entered Details:</h3>
  Name: {{user.name}} <br>
  Email: {{user.email}}
</div>

<script>
angular.module("regApp",[])
.controller("regCtrl",function($scope){
  $scope.user = {};
  $scope.show = false;
});
</script>

</body>
</html>
```

XI. Exercise

1. Differentiate one-way and two-way data binding.

One-Way Binding	Two-Way Binding
Data flows in one direction	Data flows in both directions
Model → View only	Model ↔ View
View does not update model	View and model update each other

---

2. Which are common directives used for data binding?

Common Directives:

- `ng-model`
- `ng-bind`
- `ng-init`
- Interpolation `{{ }}`

---

3. Explain `$scope` in AngularJS.

Answer:
`$scope` is an object that acts as a bridge between the **controller and the view**.
It stores data and functions that can be accessed in the HTML view.

Example:

```
$scope.name = "Student";
```

Practical No. 8: *Write a program to implement different filters in AngularJS.

IX. Conclusion

AngularJS filters are used to format and transform data before displaying it in the view.
Filters such as `uppercase`, `currency`, and `date` help in presenting data in a readable and user-friendly format.
Using filters makes AngularJS applications more dynamic and improves data presentation without changing the original data.

X. Practical Related Questions

1. Take input from user as first name and last name and display full name in capital letters.

```
<!DOCTYPE html>
<html ng-app="filterApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="filterCtrl">

First Name:
<input type="text" ng-model="firstName"><br><br>

Last Name:
<input type="text" ng-model="lastName"><br><br>

<h3>
Full Name:
{{ (firstName + ' ' + lastName) | uppercase }}
</h3>

<script>
angular.module("filterApp",[])
.controller("filterCtrl",function($scope){});
</script>

</body>
</html>
```

---

2. Accept amount from user and display it in currency format.

```
<!DOCTYPE html>
<html ng-app="currencyApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="currencyCtrl">

Enter Amount:
<input type="number" ng-model="amount">

<h3> Amount: {{ amount | currency }} </h3>
```

```
<script>
angular.module("currencyApp",[])
.controller("currencyCtrl",function($scope){});
</script>

</body>
</html>
```

---

3. Display date in MM/DD/YYYY format.

```
<!DOCTYPE html>
<html ng-app="dateApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="dateCtrl">

<h3>
Date: {{ today | date:'MM/dd/yyyy' }}
</h3>

<script>
angular.module("dateApp",[])
.controller("dateCtrl",function($scope){
  $scope.today = new Date();
});
</script>

</body>
</html>
```

XI. Exercise

1. Can we apply multiple filters to same expression? How?

Answer:
Yes, multiple filters can be applied to the same expression by using the **pipe (`|`) symbol**.

Example:

```
{{ name | uppercase | limitTo:5 }}
```

---

2. Explain how to apply filter to directive.

Answer:
Filters can be applied directly in AngularJS directives using the pipe symbol.

Example using `ng-bind`:

```
<span ng-bind="name | uppercase"></span>
```

This applies the `uppercase` filter to the `name` directive value before displaying it.

Practical No. 9: *Write a program to implement events in AngularJS.

IX. Conclusion

AngularJS events like `ng-click`, `ng-keypress`, and `ng-mousemove` help handle user actions easily.
They allow dynamic updates of data and view without directly manipulating the DOM.
Using events makes web pages interactive and responsive.

X. Practical Related Questions

1. Change the button color after clicking it.

```
<!DOCTYPE html>
<html ng-app="eventApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="eventCtrl">

<button ng-click="changeColor()" ng-style="{'background-color': btnColor}">
Click Me
</button>

<script>
angular.module("eventApp",[])
.controller("eventCtrl",function($scope){
  $scope.btnColor = "lightblue";
  $scope.changeColor = function(){
    $scope.btnColor = "lightgreen";
  };
});
</script>

</body>
</html>
```

Explanation:
The `ng-click` event changes the button color when it is clicked.

---

2. Show count of key press in the text box.

```
<!DOCTYPE html>
<html ng-app="keyApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="keyCtrl">

<input type="text" ng-keypress="countKey()">
<p>Key Press Count: {{count}}</p>
```

```
<script>
angular.module("keyApp",[])
.controller("keyCtrl",function($scope){
  $scope.count = 0;
  $scope.countKey = function(){
    $scope.count++;
  };
});
</script>

</body>
</html>
```

Explanation:
The `ng-keypress` event counts how many times a key is pressed.

---

3. Display x and y coordinates in different mouse events.

```
<!DOCTYPE html>
<html ng-app="mouseApp">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="mouseCtrl">

<div style="width:300px;height:200px;border:1px solid black"
     ng-mousemove="showXY($event)">
Move Mouse Here
</div>

<p>X Coordinate: {{x}}</p>
<p>Y Coordinate: {{y}}</p>

<script>
angular.module("mouseApp",[])
.controller("mouseCtrl",function($scope){
  $scope.showXY = function(event){
    $scope.x = event.clientX;
    $scope.y = event.clientY;
  };
});
</script>

</body>
</html>
```

Explanation:
The `ng-mousemove` event displays mouse pointer coordinates using `$event`.

XI. Exercise

1. Does AngularJS override the element's original `onmousedown` event? Justify.

Answer:
No, AngularJS does not override the original event.
AngularJS provides directives like `ng-mousedown` that **wrap the native DOM events** and handle them in an Angular-friendly way without replacing the original event behavior.

---

2. Specify the order of mouse click events.

Answer:
The correct order of mouse events is:

1. **Mouse Down**
2. **Mouse Up**
3. **Mouse Click**

Practical No. 10: Write a program to display the data in the table

IX. Conclusion

In this practical, we successfully created a dynamic table using AngularJS. The `ng-repeat` directive was used to display structured data efficiently. This experiment helps in understanding real-time data binding and table creation in AngularJS.

X. Practical Related Questions – Programs

Program 1: Display 10 Students Data in Table Format

```
<!DOCTYPE html>
<html ng-app="studentApp">
<head>
    <title>Student Table</title>
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
    <style>
        table {
            border-collapse: collapse;
            width: 60%;
        }
        th, td {
            border: 1px solid black;
            padding: 8px;
            text-align: center;
        }
        th {
            background-color: #dcdcdc;
        }
    </style>
</head>
<body ng-controller="studentCtrl">

<h3>Student Information</h3>

<table>
    <tr>
        <th>Sr. No</th>
        <th>Name</th>
        <th>Roll No</th>
        <th>Branch</th>
    </tr>
    <tr ng-repeat="s in students">
        <td>{{$index + 1}}</td>
        <td>{{s.name}}</td>
        <td>{{s.roll}}</td>
        <td>{{s.branch}}</td>
    </tr>
</table>
```

```
<script>
    var app = angular.module("studentApp", []);
    app.controller("studentCtrl", function($scope) {
        $scope.students = [
            {name:"Amit", roll:1, branch:"CS"},
            {name:"Rohit", roll:2, branch:"IT"},
            {name:"Neha", roll:3, branch:"ENTC"},
            {name:"Pooja", roll:4, branch:"CS"},
            {name:"Rahul", roll:5, branch:"IT"},
            {name:"Sneha", roll:6, branch:"CS"},
            {name:"Karan", roll:7, branch:"ENTC"},
            {name:"Priya", roll:8, branch:"IT"},
            {name:"Om", roll:9, branch:"CS"},
            {name:"Anita", roll:10, branch:"ENTC"}
        ];
    });
</script>

</body>
</html>
```

Program 2: Display Employee Data of Company

```
<!DOCTYPE html>
<html ng-app="empApp">
<head>
    <title>Employee Table</title>
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="empCtrl">

<h3>Employee Details</h3>

<table border="1">
    <tr>
        <th>Sr. No</th>
        <th>Name</th>
        <th>Designation</th>
        <th>Salary</th>
    </tr>
    <tr ng-repeat="e in employees">
        <td>{{$index + 1}}</td>
        <td>{{e.name}}</td>
        <td>{{e.designation}}</td>
        <td>{{e.salary}}</td>
    </tr>
</table>

<script>
    var app = angular.module("empApp", []);
    app.controller("empCtrl", function($scope) {
        $scope.employees = [
            {name:"Raj", designation:"Manager", salary:50000},
            {name:"Seema", designation:"Developer", salary:40000},
            {name:"Aakash", designation:"Tester", salary:30000}
        ];
    });
</script>
</body>
</html>
```

Program 3: Display List of Different Countries

```
<!DOCTYPE html>
<html ng-app="countryApp">
<head>
    <title>Country List</title>
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="countryCtrl">

<h3>Country List</h3>

<table border="1">
    <tr>
        <th>Sr. No</th>
        <th>Country Name</th>
    </tr>
    <tr ng-repeat="c in countries">
        <td>{{$index + 1}}</td>
        <td>{{c}}</td>
    </tr>
</table>

<script>
    var app = angular.module("countryApp", []);
    app.controller("countryCtrl", function($scope) {
        $scope.countries = ["India", "USA", "UK", "Japan", "Germany"];
    });
</script>

</body>
</html>
```

XI. Exercise – Answers

1. List various contextual variables available in AngularJS.

- `$index`
- `$first`
- `$last`
- `$middle`
- `$even`
- `$odd`

---

2. Which contextual variable is used to display serial number in table?

Answer:
`$index` is used to display serial number.
Example:

```
{{$index + 1}}
```

Practical No. 11: *Write a program to implement CSS to table data – odd and even rows.

IX. Conclusion

In this practical, we successfully applied CSS styling to odd and even rows of a table using AngularJS directives. This improves readability and enhances the user interface.

X. Practical Related Questions – Programs

Program 1: Display 10 Students Data with Different Styles for Odd and Even Rows

```
<!DOCTYPE html>
<html ng-app="studentApp">
<head>
    <title>Student Table</title>
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
    <style>
        table {
            border-collapse: collapse;
            width: 60%;
        }
        th, td {
            border: 1px solid black;
            padding: 8px;
            text-align: center;
        }
        .odd-row {
            background-color: #f2f2f2;
        }
        .even-row {
            background-color: #d9edf7;
        }
    </style>
</head>

<body ng-controller="studentCtrl">

<h3>Student Details</h3>

<table>
    <tr>
        <th>Sr No</th>
        <th>Name</th>
        <th>Branch</th>
    </tr>
    <tr ng-repeat="s in students" ng-class-odd="'odd-row'" ng-class-even="'even-row'">
        <td>{{$index + 1}}</td>
        <td>{{s.name}}</td>
        <td>{{s.branch}}</td>
    </tr>
</table>
```

```
<script>
var app = angular.module("studentApp", []);
app.controller("studentCtrl", function($scope) {
    $scope.students = [
        {name:"Amit", branch:"CS"},
        {name:"Rohit", branch:"IT"},
        {name:"Neha", branch:"ENTC"},
        {name:"Pooja", branch:"CS"},
        {name:"Rahul", branch:"IT"},
        {name:"Sneha", branch:"CS"},
        {name:"Karan", branch:"ENTC"},
        {name:"Priya", branch:"IT"},
        {name:"Om", branch:"CS"},
        {name:"Anita", branch:"ENTC"}
    ];
});
</script>

</body>
</html>
```

Program 2: Display Employee Data with Odd Rows in Light Green Color

```
<!DOCTYPE html>
<html ng-app="empApp">
<head>
    <title>Employee Table</title>
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
    <style>
        .odd-row {
            background-color: lightgreen;
        }
        .even-row {
            background-color: white;
        }
    </style>
</head>

<body ng-controller="empCtrl">

<h3>Employee Details</h3>

<table border="1">
    <tr>
        <th>Sr No</th>
        <th>Name</th>
        <th>Designation</th>
    </tr>
    <tr ng-repeat="e in employees" ng-class-odd="'odd-row'" ng-class-even="'even-row'">
        <td>{{$index + 1}}</td>
        <td>{{e.name}}</td>
        <td>{{e.designation}}</td>
    </tr>
</table>

<script>
var app = angular.module("empApp", []);
app.controller("empCtrl", function($scope) {
    $scope.employees = [
```

```
        {name:"Raj", designation:"Manager"},
        {name:"Seema", designation:"Developer"},
        {name:"Aakash", designation:"Tester"},
        {name:"Neha", designation:"HR"},
        {name:"Om", designation:"Clerk"},
        {name:"Ravi", designation:"Engineer"},
        {name:"Anita", designation:"Admin"},
        {name:"Kiran", designation:"Designer"},
        {name:"Pooja", designation:"Support"},
        {name:"Rahul", designation:"Supervisor"}
    ];
});
</script>

</body>
</html>
```

Program 3: Display Staff Data with Even Rows in Light Green Color

```
<tr ng-repeat="staff in staffList"
    ng-class-odd="'odd-row'"
    ng-class-even="'even-row'">
.odd-row {
    background-color: white;
}
.even-row {
    background-color: lightgreen;
}
```

XI. Exercise – Answers

1. Which pseudo-class in CSS is used to apply different style to odd or even rows of table?

Answer:

- `:nth-child(odd)`
- `:nth-child(even)`

Example:

```
tr:nth-child(odd) {
    background-color: #f2f2f2;
}
tr:nth-child(even) {
    background-color: #d9edf7;
}
```

---

2. Give different ways to format even and odd rows in the table in AngularJS.

Answer:

1. Using `ng-class-odd` and `ng-class-even`
2. Using CSS pseudo-classes `:nth-child(odd)` and `:nth-child(even)`

3. Using `$index` inside `ng-repeat` with conditional classes
4. Using external CSS file with predefined row styles

IX. Conclusion

In this practical, we implemented different types of AngularJS controllers such as Basic and Nested controllers. We understood how controllers manage data using the $scope object and connect the view with application logic. Thus, AngularJS controllers help in building dynamic and structured web applications.

Practical No. 12: * Write programs for implementation of different methods of AngularJS Controllers

X. Practical Related Questions

---

1. Write a program to display student details (name, roll no, marks) using a controller.

```
<!DOCTYPE html>
<html ng-app="myApp">
<body ng-controller="StudentCtrl">

<h2>Student Details</h2>
Name: {{name}} <br>
Roll No: {{rollno}} <br>
Marks: {{marks}}

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

<script>
var app = angular.module("myApp", []);

app.controller("StudentCtrl", function($scope) {
   $scope.name = "Ganesh";
   $scope.rollno = 101;
   $scope.marks = 85;
});
</script>

</body>
</html>
```

---

2. Write a program where one controller is nested inside another (Parent–Child).

```
<!DOCTYPE html>
<html ng-app="myApp">
<body>

<div ng-controller="ParentCtrl">
   <h2>Parent Message: {{parentMsg}}</h2>

   <div ng-controller="ChildCtrl">
      <h3>Child Message: {{childMsg}}</h3>
      <h3>Inherited from Parent: {{parentMsg}}</h3>
   </div>
</div>

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

<script>
var app = angular.module("myApp", []);

app.controller("ParentCtrl", function($scope) {
   $scope.parentMsg = "Hello from Parent";
});

app.controller("ChildCtrl", function($scope) {
   $scope.childMsg = "Hello from Child";
});
</script>

</body>
</html>
```

XI. Exercise

1. What is the role of the $scope object in a controller?

$scope is an object that connects the controller with the HTML view.

Role of $scope:

- Stores data (variables)
- Stores functions
- Passes data from controller to view
- Enables two-way data binding

---

2. How do you attach variables and functions to the $scope object?

By defining them inside the controller using $scope.

Example:

```
app.controller("DemoCtrl", function($scope) {

   $scope.name = "Ganesh";   // Variable

   $scope.showMsg = function() {   // Function
      alert("Welcome " + $scope.name);
   };

});
```

Here:

- $scope.name → Variable
- $scope.showMsg() → Function

Both can be accessed in HTML using AngularJS expressions.

Practical No. 13: * Write programs to demonstrate use of controllers in external files

IX. Conclusion

In this practical, we demonstrated AngularJS controllers using external JavaScript files. We learned that separating controllers from HTML makes code modular, reusable, and easy to maintain. Hence, external controllers are useful for developing structured web applications.

X. Exercise

---

1 Program: Display Message using External Controller

File: 1.js

```
angular.module("myApp", [])
.controller("MessageCtrl", function($scope) {
   $scope.message = "Hello from External Controller!";
});
```

File: 1.html

```
<!DOCTYPE html>
<html ng-app="myApp">
<body ng-controller="MessageCtrl">

<h2>{{message}}</h2>

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
<script src="1.js"></script>

</body>
</html>
```

---

2 Program: Student Table with orderBy Filter

File: 2.js

```
angular.module("myApp", [])
.controller("StudentCtrl", function($scope) {
    $scope.students = [
        {name: "Rohit", marks: 80},
        {name: "Amit", marks: 85},
        {name: "Sneha", marks: 90}
    ];
});
```

File: 2.html

```
<!DOCTYPE html>
<html ng-app="myApp">
<body ng-controller="StudentCtrl">

<h2>Student List (Sorted by Name)</h2>

<table border="1">
<tr>
    <th>Name</th>
    <th>Marks</th>
</tr>

<tr ng-repeat="s in students | orderBy:'name'">
    <td>{{s.name}}</td>
    <td>{{s.marks}}</td>
</tr>
</table>

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
<script src="2.js"></script>

</body>
</html>
```

---

3 Program: Uppercase & Lowercase Filters

File: 3.js

```
angular.module("myApp", [])
.controller("TextCtrl", function($scope) {
   $scope.text = "AngularJS External Controller";
});
```

File: 3.html

```
<!DOCTYPE html>
<html ng-app="myApp">
<body ng-controller="TextCtrl">

<h3>Original: {{text}}</h3>
<h3>Uppercase: {{text | uppercase}}</h3>
<h3>Lowercase: {{text | lowercase}}</h3>

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
<script src="3.js"></script>

</body>
</html>
```

XI. Practical Related Question

Q. What is the difference between an internal and an external controller?

Internal Controller	External Controller
Written inside HTML	Written in separate JS file
Less modular	More modular
Difficult to maintain	Easy to maintain
Used for small apps	Used for large apps